

Track C: Making the Two Halves Talk

Your job this week, in one sentence

Delete the fake reply and make the **real** AI model answer inside the app you already built.

Why this week is different

For two weeks the two halves of this project have never met. One of you built a dashboard with a pretend engine behind it. The other built a real engine with no dashboard on it. Both work. Neither has ever been plugged into the other.

This week you don't design anything new. You connect what exists, and you delete a lie — that hardcoded fake reply. That sounds smaller than it is. Two things stand in the way, and both of them are famous for eating an afternoon.

A few words you'll see, explained simply

- **Origin** — an address is not just the host, it's the *protocol + host + port* together. So `localhost:5173` and `localhost:5000` are two **different** origins, even though they're the same computer. This is the whole reason this week is hard.
- **CORS** — a browser safety rule. When your page asks a *different* origin for something, the browser refuses to hand you the answer unless that server explicitly said "this page is allowed." Read that again: it's the **browser** refusing, not the server. Almost everyone gets this backwards at first, and backwards makes the error message impossible to understand.
- **Preflight** — for some requests, the browser doesn't send your request at all. First it sends a tiny separate question ("may I?") and waits. If it doesn't like the answer, your real request is **never sent** — so nothing shows up in the backend's log, and it looks like your code didn't run.
- **Stream (SSE)** — instead of one big answer at the end, the server holds the connection open and pushes the answer down it in small pieces. Your backend already does this.
- **React Query** — a library that handles fetching data *and* remembering it, so you stop hand-writing "is it loading / did it fail / refresh it now" logic yourself. It's already installed in the project. Nobody ever switched it on.
- **Mutation** — a request that *does something* rather than just *reads something*. Sending a chat message is a mutation. Asking "are you alive?" is not.

Part 1 — the backend half

1. Add a `/health` route. A tiny route that answers `{"status": "ok"}`. That's it. It exists so the frontend can ask "are you alive?" without sending a whole chat message and waiting 20 seconds for an answer.

2. Give the Ollama client a 5-second timeout. One extra line where the client is created. Skip it and you get a horrible bug: if Ollama isn't reachable, `/health` never answers *at all* — it doesn't fail, it just hangs forever, and the app sits there spinning. A health check that can hang is worse than no health check.

3. Turn on CORS — but not yet. You need one line and one package (`flask-cors`). **Wait until the frontend actually hits the error before you add it.** The whole point is to see the thing break, read the real error message, and then watch one line fix it. If you add it early, nobody learns what it does and you'll be confused by it forever.

4. Add two headers to the streaming reply. `Cache-Control: no-cache` and `X-Accel-Buffering: no`. Some networks and proxies will otherwise quietly collect all your little pieces and hand them over in one lump — which technically works, but destroys the whole point of streaming.

5. Test with `curl` before involving a browser. Same habit as last week, same reason: when something breaks you want to know which half broke. Add `-N` to your `curl` command so `curl` doesn't do its own buffering and lie to you. You should see pieces arrive one at a time, not a single block at the end.

Part 2 — the frontend half

1. Switch React Query on. Wrap the app in its provider at the very top, and turn on its devtools panel while you're there. This is pure setup, five minutes, no logic.

2. Make the status badge tell the truth. Look at the little green "Online" dot in the header. It is **hardcoded**. It says "Online" even with the entire backend switched off. Right now your app lies to whoever uses it. Point it at `/health` instead and give it a refresh interval, so it checks every few seconds by itself.

3. Watch it fail. This is the point, not an accident. Your first real request to the backend will be blocked. Open the browser devtools, go to Console, and actually read the red message. Then look at the Network tab and ask the useful question: *did my request even leave the building?* Now the backend half turns CORS on, and you watch it go green.

4. Replace the fake reply with the real one. Your backend sends the answer as a series of small pieces. You need to open the connection, read pieces as they arrive, pull the actual text out of each one, and add it to the message on screen. Two things about this that trip everyone up:

- One "read" does **not** equal one message. The network splits things wherever it likes, so a piece can arrive holding one and a half messages. You have to hold onto the leftover half and wait for the rest. If you skip this you get a crash that makes no sense.
- Your backend puts the literal text `data:` in front of every message. You have to strip that off before you can read what's inside. Open `main.py` and find the line that adds it — seeing both ends of the same format is the moment this stops feeling like magic.

Then **delete the fake reply function**. Enjoy it. That function is the reason this project worked before it did anything.

5. Make Send a mutation. Sending a message is a "do something" request, so it's a mutation, not a query. Doing it this way hands you "is it currently running?" and "did it fail?" for free, and you get to delete the flag you were tracking by hand.

6. Make the Stop button work again. Careful — this one breaks silently. The old Stop button cancelled timers, and once the fake reply is gone there are no timers to cancel. The button will still be there, still look fine, and do **absolutely nothing**. Real requests are cancelled a different way (`AbortController`). If you run out of time, hide the button rather than leaving a dead one on screen.

7. Two leftovers from last week. The friendly empty-state message is currently **two completely empty lines** — it was on the checklist and never got written. And nobody ever checked the app on a phone-sized screen; the chat box has a fixed height that doesn't fit.

Your AI helper will get these five things wrong

You have an AI to help you this week, and it will be genuinely useful. It will also walk you confidently into these specific holes, because it doesn't know how *your* project is built. When it does, you'll know more than it does about this project.

1. **It will tell you to use EventSource.** Every tutorial on the internet uses EventSource for streaming, so the AI will suggest it. **EventSource can only do GET requests. Your /chat is a POST.** It cannot work, no matter how you configure it. If you don't know this in advance you can lose an hour to it.
2. **It will offer to "fix" CORS by adding a proxy to Vite.** That does work, by pretending both halves share one address. It also means you never learn what actually broke, and you'll be helpless the next time it happens somewhere you can't add a proxy. Fix it on the server this week.
3. **It will write the status badge wrong.** It'll check whether the last answer said "ok". The trap: React Query deliberately **keeps the last good answer** after a request fails — stale data is usually better than a blank screen. So with the backend switched off, the badge keeps happily

saying "Online". You also have to check whether the request *errored*, and switch retries off so it reacts quickly.

4. **It may "tidy up" a setting in the text decoder that looks pointless.** There's an option that tells it "more bytes are coming, don't panic yet". Remove it and everything works fine — right up until someone types a Romanian letter. *ă*, *î* and *ș* don't fit in one byte, so they get split down the middle and turn into garbage. Keep it.
5. **It will probably tell you the preflight fails with a 405 error.** It doesn't. It comes back 200, looking perfectly fine, and the browser rejects it anyway because one header is missing. If you go hunting for a 405 you'll never find one.

Ask it to explain, not just to write. "Why does my request get blocked when the server returned 200?" teaches you something. "Write me a chat component" teaches you nothing and gives you code you can't debug. You will have to debug it.

How you'll know you're done

- ☐ Asking the backend `/health` with curl gets an "ok" answer back.
- ☐ The badge is green. Switch the backend off — it goes red within about 5 seconds. Switch it back on — green returns.
- ☐ You type a real message and get a real answer from the actual model.
- ☐ The answer arrives in pieces, word by word — not one block at the end.
- ☐ Stop, pressed halfway through, actually stops it and keeps the text so far.
- ☐ A question with Romanian letters in it ("*ce înseamnă recursivitate?*") comes back readable, not corrupted.
- ☐ The empty chat screen says something friendly instead of nothing.
- ☐ The app is usable on a phone-sized screen.

What you don't need to worry about yet

- Real file uploads — the attach button still just needs to *look* like it worked
- Actually sending attached files to the model
- Deployment, Hugging Face, putting it on the internet
- Logging in, saving conversations, having more than one chat
- Speed. It's running on a normal processor with no graphics card. Ten to twenty seconds for a full answer is completely normal and is not something you broke. It's also exactly why streaming matters — the wait *feels* far shorter when you can see words appearing.

A realistic warning about time

Two parts of this will take longer than they look. The CORS error is a five-second fix that takes forty minutes to understand, and reading the streamed pieces correctly is genuinely fiddly. If you only finish one thing today, finish the streaming — real words from a real model appearing on screen is the entire point of the week. The empty-state text and the phone layout can wait.